Chapter 6

Data augmentation, Loss functions

Reading

1. Bishop Chapter 5.5.3, 4.3
2. Bishop DL Chapter 6.4, 9.1
3. Goodfellow Chapter 7.4

6.1 Data augmentation

In the previous chapter, we looked at convolutions as a way to reduce the parameters in a deep network, but more importantly as a way of building equivariance/invariance to translations. There are a lot of nuisances other than translation that do not have a group structure which precludes operations such as convolutions that we can perform to generate equivariance/invariance.

In this section, we will discuss techniques to build invariance to nuisances that are more complex than just translations, these techniques will seem brute-force but they also allow us to handle more complex nuisances. The main trick is to *augment* the data, i.e., create variants of each input datum in some simple way such that we *know* that its label is unchanged. If our original dataset is $D = \left\{(x^i, y^i)\right\}_{i=1,\ldots,n}$ we create an augmented dataset

$$T(D) := \left\{(T(x^i), y^i)\right\}_{i=1,\ldots,n} \cup D. \tag{6.1}$$

where T is some operation of our choice. We have therefore expanded the number of samples in the training dataset to $2n$ instead of the original

n. Effectively, data augmentation is a technique to create a dataset that is sampled from some other data distribution P than the original one.

6.1.1 Some basic data augmentation techniques

The most popular data augmentation techniques are setting T to be changes in brightness, contrast, cropping the image to simulate occlusions, flipping the image horizontally or vertically, jittering the pixels of the input image to simulate noise in the CCD of the camera/weather, padding the image which changes the borders of the input image, warping the image using a projection that simulates the same picture taken from a different viewpoint, thresholding the RGB color channels, zooming into an image to simulate changes in the scale etc.

You can see these operations at https://fastai1.fast.ai/vision.transform.html#List-of-transforms.

⚠ FastAI is a wrapper on top of PyTorch and is an excellent library to learn for doing your course projects.

6.1.2 How does augmentation help?

A number of such augmentations are applied to the input data while training a deep network. This increases the number of samples n we have for training but note that different samples share a lot of information, so the effective novel samples has not increased by much. Let us get an idea of when augmentation is useful and when it is not. Consider a regression

and classification problem as shown below.

Figure 6.1: Cows live in many different parts of the world. A classifier that also uses background information to predict the category is likely to make mistakes when it is run in a different part of the world. Augmenting the input dataset on the left by replacing the background to include a mountain or a city is therefore a good idea if we want to run the classifier in a different part of the world. This will also force the classifier to *ignore* the background pixels when it classifies the cow, in other words the classifier is forced to become invariant to backgrounds by brute-force showing it different backgrounds.

In essence, data augmentation forces the model to tackle a larger dataset than our original dataset. The model is forced to learn what nuisances the designer would like it to be invariant to. Compare this to the previous chapter: by replacing fully-connected layers with convolutions and pooling we made the model invariant to translations. In principle, we could have trained a fully-connected deep network on a very large augmented dataset with translated objects. In principle, this would make the fully-connected network invariant to translations as well.

6.1.3 What kind of augmentation to use when?

In the example with regression, we saw that the regressor on the augmented data was essentially linear and had much less discriminative power than a polynomial regressor. This was of course by design, we chose to augment the data. If the test data for the problem came from the polynomial instead of our augmented distribution, the new classifier will perform poorly.

Figure 6.2: The second panel shows the original scene with a mirror flip (i.e., across the horizontal axis) while the third panel shows the original scene after a water reflection (i.e., flip across the vertical axis). The latter is an image that is very unlikely to occur in the real world, so it is not a good idea to use it for training the model.

By being invariant to a larger set of nuisances than necessary, we are wasting the parameters of the model and risk getting a large error if the test data was not from the augmented distribution. By being invariant to a smaller set of nuisances than necessary, we are risking the situation that the test data will have some new nuisances which the classifier will perform poorly on. It is important to bear in mind that we do not always know what nuisances the model should be invariant to, the set of transformations in data augmentations necessarily depends—often critically—upon the application.

❓ If you are building a classifier for detecting cars, motorbikes, people etc. for autonomous driving application, do you want to be the invariant to rotations?

Data augmentation requires a lot of domain expertise and often plays a huge role in the performance of a deep network. You should think about what kind of augmentations you will apply to data for speech processing, or for data from written text.

❓ Think of what kind of data augmentations you would use for a language processing model. What are the equivariances that we might wish to capture?

6.2 Loss functions

We next discuss the various loss functions that are typically used for training neural networks. As usual, we are given a dataset

$$D = \left\{(x^i, y^i)\right\}_{i=1,\ldots,n}.$$

6.2.1 Regression

MSE loss. If the labels are real-valued $y^i \in \mathbb{R}$, e.g., we are predicting the price of housing in Boston given features of the houses (like you did in HW 0), we are solving a regression problem and the loss function to use for a deep network is also simply the regression loss.

$$\ell_{\text{mse}}(w) := \frac{1}{2}\left(f(x; w) - y\right)^2 \tag{6.2}$$

If you think about it carefully, it seems silly to add different dimensions of the input x using the weights w. Consider the case of $x = [\text{miles/gallon}, \text{number of other people with the same car}, \text{price of the car}]$. The three elements of x are in totally different units and totally different scales. A popular trick to make things a bit more uniform for regression

is take a logarithmic transformation of the input, i.e., fit a model to $\log x$ using the loss

$$\frac{1}{2}\left(f(\log x; w) - y\right)^2;$$

we can compute the logarithm element-wise for vector valued inputs.

Huber loss. The square-residual loss in Eq. (6.2) works in most cases but it does not work well if there are outliers in the data. Outliers are data in the training set that are noisy or did not come from the true model. In such cases, we can use the Huber loss. If the residual is $r = f(x; w) - y$, the Huber loss is

$$\ell_{\text{huber}}(w; \delta) = \begin{cases} \frac{1}{2}|r|^2 & \text{if } |r| \leq \delta \\ \delta\left(|r| - \frac{1}{2}\delta\right) & \text{else.} \end{cases} \tag{6.3}$$

Observe that this does not penalize the model egregiously if the predictions are bad ($|r| \geq \delta$) for a particular datum. Doing so prevents the outliers from biasing the loss towards themselves and ruining the residuals for the other data.

⚠ We can perform regression in a clever way: first set all weights $w_i = 0$ and iteratively allow a subset of the weights (say the ones that improve the residuals the most) to become non-zero; non-zero weights are fitted using ℓ_{mse}. This is known as forward selection. Backward selection starts with weights w^* which minimize ℓ_{mse} and iteratively prune the weights. Both forward and backward selection are techniques to fit a model w^* with sparse weights.

MAE loss. The absolute-error loss (or ℓ_1)

$$\ell_{\text{mae}}(w) = |f(x; w) - y| \tag{6.4}$$

has a similar motivation: it does not penalize the residual on the outliers.

⚠ Using a subset-selection technique or the ℓ_{mse} loss with ℓ_1 regularization on the weights

$$\frac{1}{2n}\sum_{i=1}^{n}\left(f(\log x^i; w) - y^i\right)^2 + \lambda\|w\|_1$$

leads to sparse weights w^*. This makes the model more interpretable than a model fitted using ℓ_{mse} loss. This is easy to understand for linear models: input dimensions corresponding to weights w_i^* that are zero do not take part in making predictions. So one may answer questions of the form "is variable x_i a relevant predictor of the target y".

Variable importance. For linear models, another way to answer the same question is to fit two models, one with w_i fixed to zero and all other weights fitted using the MSE loss Eq. (6.2) and another model without fixing w_i; the difference between the average square residuals in the two cases is a measure of how important the feature x_i is for the prediction. These techniques are called variable importance methods. We can also undertake the same program for nonlinear models on non-image based data.

Quantile loss. The quantile loss is another simple trick to make the model more robust to outliers and get more information from the model than simply the prediction $f(x; w)$. Observe that if we have targets Y that are random variables with cumulative distribution function $F(y) = \mathrm{P}(Y \leq y)$, the τth quantile of Y is given by

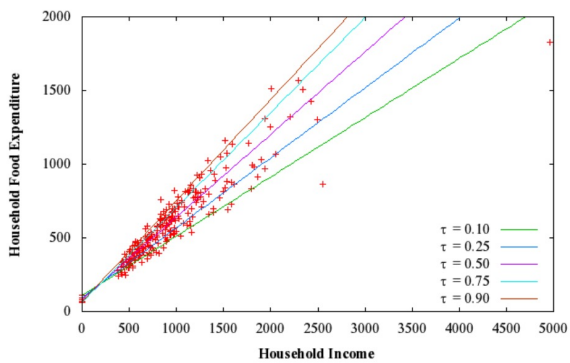
$$Q_Y(\tau) = F^{-1}(\tau) = \inf\{y : F(y) \geq \tau\}$$

for $\tau \in (0, 1)$. We now learn a predictor for $Q_Y(\tau) = f(x; w)$. It turns out (you can try to prove this) that this corresponds to the loss function

$$\ell_{\text{quantile}}(w; \tau) = \begin{cases} r(\tau - 1) & \text{if } r < 0 \\ r\tau & \text{else.} \end{cases} \tag{6.5}$$
$$= r\left(\tau - \mathbf{1}_{\{r<0\}}\right).$$

where $r = y - f(x; w)$ is the residual. A standard technique is to fit multiple models using the quantile loss for different quantiles, say $\tau = 0.25, 0.5, 0.75$ and give multiple predictions of the target $f(x; w^\tau)$. A typical example of quantile linear regression looks as follows.

⚠ The quantile loss is also called the pinball loss. Unlike the regression loss, it is highly asymmetric around the origin. If $r > 0$, we are penalizing the model by $\tau|r|$, and if $r < 0$, i.e., if we predict something that is larger than the true y, then we are penalizing the model by $(1 - \tau)|r|$.



6.2.2 Classification: Cross-Entropy loss

We next discuss the case when the targets are categorical and we wish to train a discriminative model that classifies the input into one of these m categories

$$y \in \{1, \ldots, m\}.$$

One hot encoding.

An alternative representation of the targets in classification is so-called the *one-hot* encoding where y is transformed to

$$\text{one-hot}(y) = e_y \in \mathbb{R}^m;$$

the vector e_y has a 1 at the yth element and zeros everywhere else. The notation e_y denotes the yth row of the identity matrix $I_{m \times m}$.

Predicting class probabilities.

Instead of using the regression loss by treating y as a real-valued quantity, it is more natural to predict the log-probability $\log p(k \mid x)$ for every category k using weights w and predict the category using

$$f(x; w) = \operatorname*{argmax}_k \ \log p_w(k \mid x). \tag{6.6}$$

Just like we denoted the raw predictions of the model by $\hat{y}$ in linear/logistic regression, we will denote

$$\mathbb{R}^m \ni \hat{y} = v^\top \sigma\left(S_L^\top \ldots \sigma\left(S_2^\top\ \sigma(S_1^\top x)\right)\ldots\right) \tag{6.7}$$

where $v \in \mathbb{R}^{p\times m}$. As we saw in Chapter 4, $\hat{y}$ are also called logits. Observe that the logits $\hat{y}$ are simply vectors in $\mathbb{R}^m$. How can we transform these logits to get $\log p_w(k \mid x)$ for all $k \in \{1, \ldots, m\}$ as the output of the model?

Logistic loss.

Linear logistic regression has a scalar output $\hat{y} \in \mathbb{R}$ which is interpreted as the log-odds of the class probabilities

$$\log \frac{p(y=1 \mid x)}{p(y=0 \mid x)} = w^\top x. \tag{6.8}$$

This expression can be rewritten as $p(1 \mid x) = \text{sigmoid}(\hat{y})$. The likelihood of data x under this model for $y^i \in \{0, 1\}$ is

$$p_w(\{(x^1, y^1), \ldots, (x^n, y^n)\}) = \prod_{i=1}^n p_w(1 \mid x^i)^{y^i} p_w(0 \mid x^i)^{1-y^i}.$$

Maximizing this probability (MLE) is the same as minimizing the log-probability

$$\begin{aligned}\ell_{\text{logistic}}(w) &= -\log p_w(\{(x^1, y^1), \ldots, (x^n, y^n)\}) \\ &= -\sum_{i=1}^n y^i \log p_w(1 \mid x^i) + (1-y^i)\log p_w(0 \mid x^i)\end{aligned} \tag{6.9}$$

In other words, the logistic loss is simply maximum-likelihood estimation for the model Eq. (6.8).

❓ We saw a different expression for the logistic loss in Chapter 3

$$\ell_{\text{logistic}}(w) = \log\left(1 + e^{-y\ \hat{y}}\right).$$

What is the difference?

Binary Cross-Entropy loss.

Let us turn back to neural networks and multi-class classification. Imagine if each logit of a neural network in Eq. (6.7) acts independently, i.e., it predicts whether there is class k in this input or not without paying heed to what the other logits predict. This is not very prudent, for instance, if we know beforehand that there is only one object in the input image, then such a classifier is likely to have lots of false positives. Nevertheless, observe that this is exactly like running m independent binary logistic classifiers with the same feature $h^L \in \mathbb{R}^p$. We can write the loss for such a classifier succinctly as

$$\ell_{\text{bce}}(w) = -\sum_{k=1}^m \text{one-hot}(y)_k \log p_w(k \mid x). \tag{6.10}$$

If the ground-truth labels y^i are such that there is only one class in each input image, all entries of one-hot(y^i) at other categories will be zero, so this loss penalizes only the output of one of the m independent logistic classifiers.

6.2.3 Softmax Layer

Observe that our classifier which employs m binary logistic classifiers for predicting all the categories independently does not predict a valid probability distribution because

$$\sum_{k=1}^{m} p_w(k \mid x)$$

is not always equal to 1. We can however posit that the model predicts logits $\hat{y}$ that are proportional to the log-probabilities

$$\begin{aligned} &\log p_w(k \mid x) \propto \hat{y}_k \\ \implies &p_w(k \mid x) = \frac{e^{\hat{y}_k/T}}{\sum_{k'=1}^{m} e^{\hat{y}_{k'}/T}}. \end{aligned} \tag{6.11}$$

The result $p_w(k \mid x)$ is a valid distribution on k because it sums up to 1. This operation, namely taking the logits $\hat{y}$ and constructing a probabilities out of them is called as a softmax operator. The constant T in Eq. (6.11) is called the temperature. A large value of T results in a smoother probability distribution $p_w(k \mid x)$ because the individual values of the logits matter less. A small value of T results in a very large weight due to the exponent on the largest logit and the distribution $p_w(k \mid x)$ is therefore highly spiked. The temperature is set to 1 by default in PyTorch.

The cross-entropy loss is now simply the maximum-likelihood loss after the softmax operation

$$\begin{aligned} \ell_{\text{ce}}(w) &= -\sum_{k=1}^{m} \text{one-hot}(y)_k \log p_w(k \mid x) \\ &= -\frac{\hat{y}_y}{T} + \log\left(\sum_{k'=1}^{m} e^{\hat{y}_{k'}/T}\right). \end{aligned} \tag{6.12}$$

Observe that the logit corresponding to the true class $\hat{y}_y$ is being pushed higher; at the same time, if the logits of the incorrect classes are large they are being pulled *down* in the summation. This is an important point to keep in mind: the cross-entropy loss after softmax affects *all* logits, not just the logit of the correct class.

6.2.4 Label smoothing

The correct logit in Eq. (6.12) is encouraged to go to $+\infty$ while the incorrect logits are encouraged to go to $-\infty$. This can lead to dramatic over-fitting when the number of classes m is very large. Label smoothing

⚠ You will often see people calling

$$\log \sum_{k'=1}^{m} e^{\hat{y}_{k'}/T}$$

as the "softmax" of vector $\hat{y}$. This is actually a more appropriate usage of the word because

$$\log \sum_{k=1}^{m} e^{\hat{y}_k/T} \approx \max_k \hat{y}$$

if one of the entires of $\hat{y}$ is much larger than the others, or if $T \to 0$. We will however use the word "softmax" to refer to the operation of transforming $\hat{y}$ into $p_w(k \mid x)$ because we do not have any need for this softened version of the max operator.

is a trick that alleviates the problem: instead of using a one-hot encoding of the true label y, it uses the encoding

$$\text{label-smoothing}(y)_k = \begin{cases} 1-\epsilon & \text{if } k = y, \\ \frac{\epsilon}{m-1} & \text{else.} \end{cases} \tag{6.13}$$

The cross-entropy loss with this new encoding is now

$$\begin{aligned} \ell_{\text{label-smoothing-ce}}(w) &= -\sum_{k=1}^{m} \text{label-smoothing}(y)_k \log p_w(k \mid x) \\ &= -(1-\epsilon)\log p_w(y \mid x) - \frac{\epsilon}{m-1}\sum_{k \neq y} \log p_w(k \mid x) \end{aligned} \tag{6.14}$$

If you take the derivative of this loss with respect to $\hat{y}$ you will see that the value of $\hat{y}$ that minimizes the loss is

$$\hat{y}_k^* = \begin{cases} \log\left((m-1)(1-\epsilon)/\epsilon\right) + \alpha & \text{if } k = y \\ \alpha & \text{else.} \end{cases} \tag{6.15}$$

where α is an arbitrary real number. Notice that logits for both the correct and the incorrect classes are finite in this case, they no longer blow up to infinity.

6.2.5 Multiple ground-truth classes

If there are multiple classes that are all present in the input image, i.e., if the ground truth data has multiple labels, we can easily use the vector

$$\text{multi-hot}(y) = \sum_k e_k$$

for all the present classes k and set

$$\ell_{\text{bce}}(w) = -\sum_{k=1}^{m} \text{multi-hot}(y)_k \log p_w(k \mid x) \tag{6.16}$$

in the BCE loss. We can also use this trick in the cross-entropy loss after the softmax operator but it will not work well because the softmax operator is designed to amplify only the largest logit in $\hat{y}$; if we tried the network would still be incentivized to predict only one class instead of all classes.

Bibliography